

COMMUNITY HEALTH SURVEY SYSTEM

DevOps Mini-Project — Week 3 Deliverable

Requirements, Architecture & Technology Setup

This document is the Week 3 deliverable for the Community Health Survey System DevOps mini-project. It provides the SRS summary, use-case diagram, architecture diagram, data model, API list and a working local setup guide, reflecting the local Spring Boot foundation that has been implemented so far.

1. Software Requirements Specification (SRS) Summary

1.1 System Overview

The Community Health Survey System is a small web-based application for recording, viewing, updating and searching community health survey records. The system provides a basic role-based workflow and a dashboard for summarizing survey progress.

1.2 Target Users

- Surveyor
- Administrator
- Health Officer
- Community Respondent

1.3 Functional Requirements

ID	Requirement
FR-1	The system shall allow a Surveyor to create a new survey record.
FR-2	The system shall allow authorized users to view survey records.
FR-3	The system shall allow survey information to be updated.
FR-4	The system shall allow users to search survey records.
FR-5	The system shall support the status workflow: DRAFT → SUBMITTED → VERIFIED → CLOSED.
FR-6	The system shall display total records and counts by status on a dashboard.
FR-7	The system shall provide role-specific access for Surveyor, Administrator and Health Officer.

1.4 Survey Data

Each survey record contains: ID, Name, Age, Gender, Location, Health Condition, Survey Date, Status, and Creator.

1.5 Non-Functional Requirements

- Simple web-based interface
- Relational database storage
- Maven-based build
- Automated testing (JUnit + Selenium)
- CI/CD using Jenkins
- Docker-based deployment
- Ansible-based provisioning

1.6 Constraints

- MVP scope; 15-week academic timeline
- Dummy/anonymized health data only — no real patient medical records
- No mobile application, no advanced AI/ML prediction, no Kubernetes

1.7 Current Week 3 Status

- Spring Boot project created
- Maven build working
- MySQL connection working
- Survey entity, repository, service and controller created
- Create survey functionality working
- View survey functionality working

This stays closely aligned with the approved Week 1 scope rather than introducing new requirements.

2. Use-Case Diagram

The diagram below shows how each actor interacts with the system's core use cases.

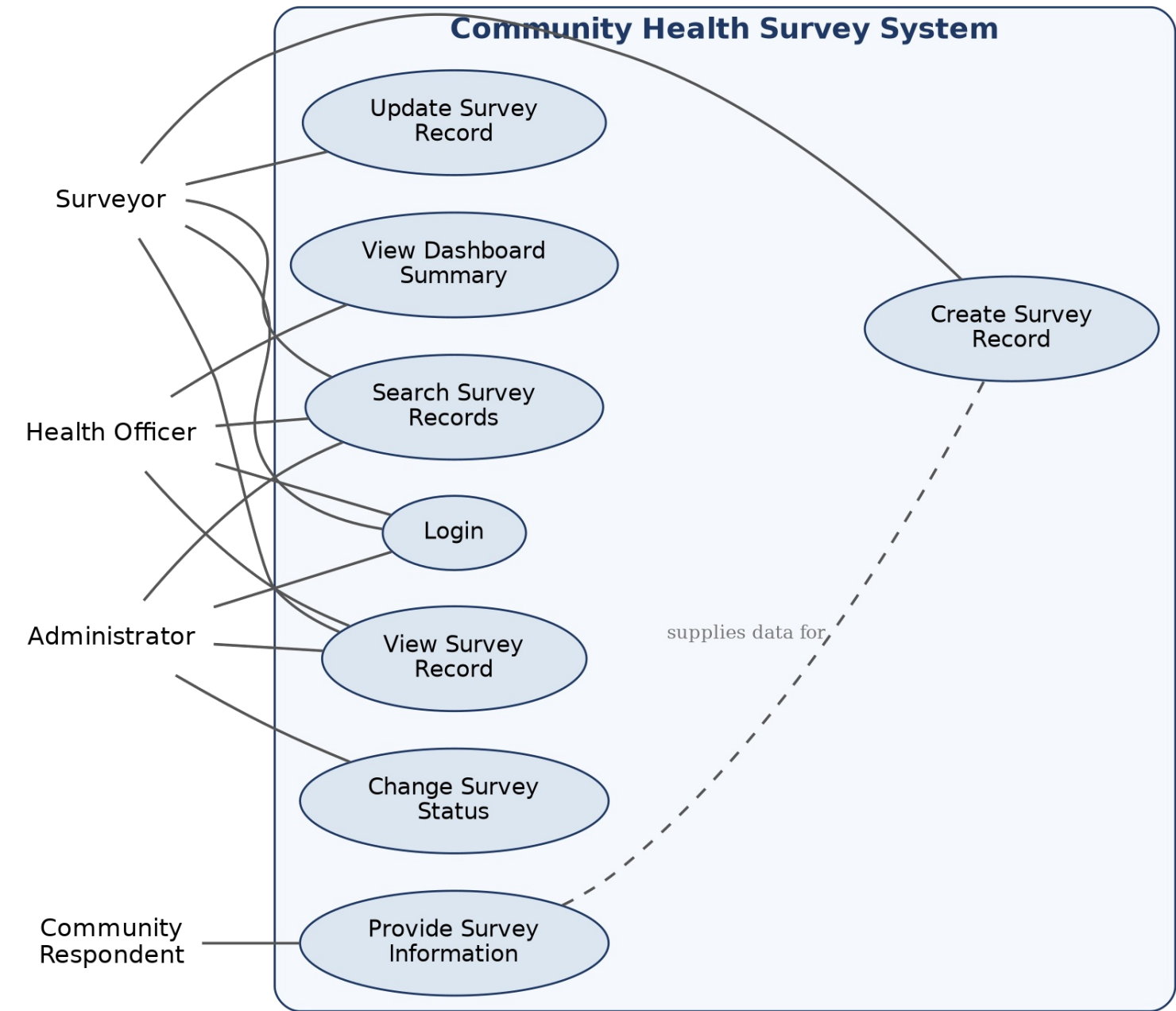


Figure 1: Use-case diagram — actors and system use cases

3. Architecture Diagram

The application follows a standard layered Spring Boot architecture (Controller → Service → Repository → Database), supported by a testing quality gate and a build/delivery pipeline.

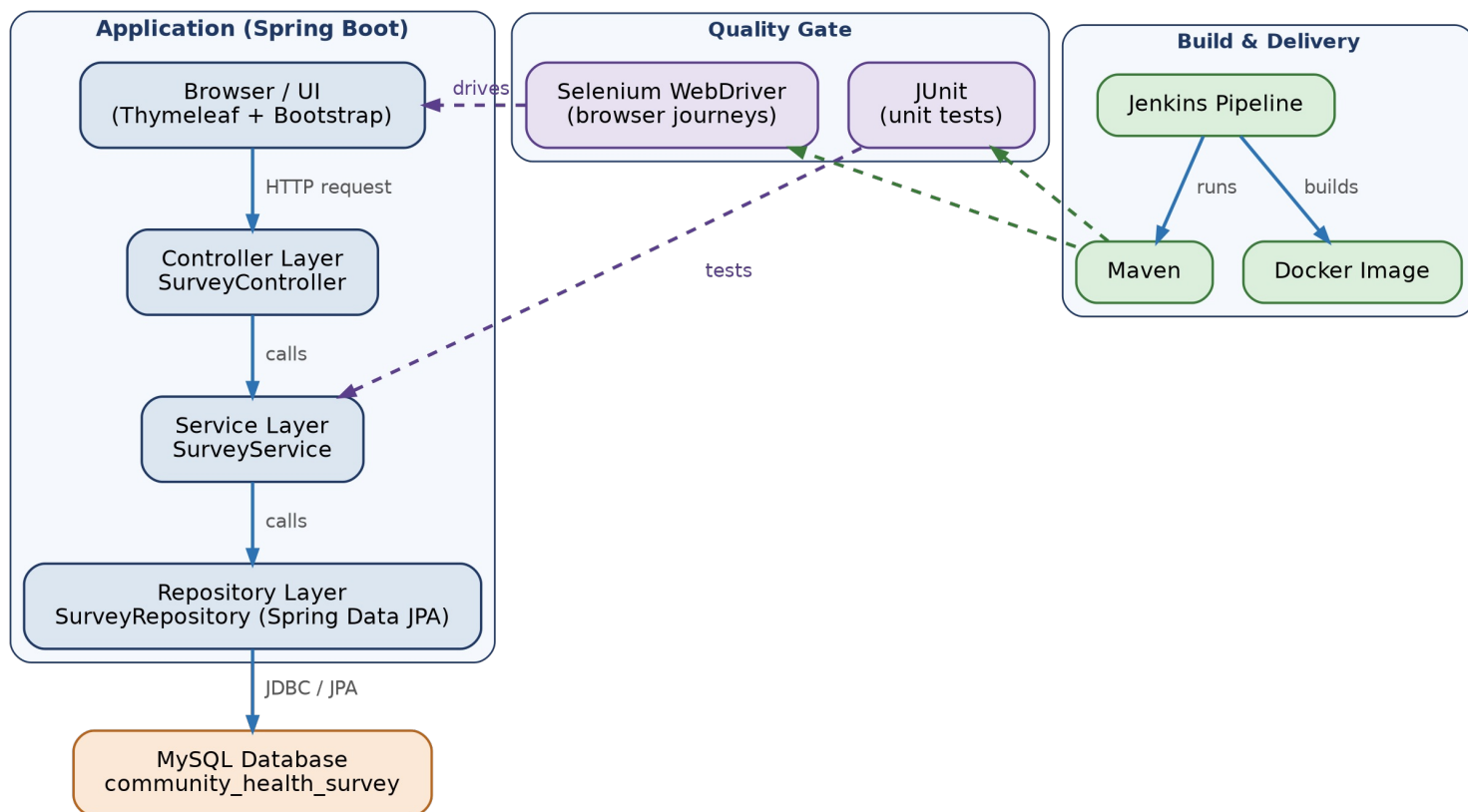


Figure 2: Layered application architecture with testing and build/delivery context

4. Data Model

The relational data model currently has two tables: `surveys` (the core entity implemented so far) and `users` (part of the approved target model for role-based login, not yet implemented).

users	
id	BIGINT (PK)
username	VARCHAR
password	VARCHAR
role	VARCHAR

creates

surveys	
id	BIGINT (PK)
name	VARCHAR
age	INTEGER
gender	VARCHAR
location	VARCHAR
health_condition	VARCHAR
survey_date	DATE
status	VARCHAR
<i>created_by</i>	BIGINT (FK)

Figure 3: Entity-relationship overview — users and surveys

4.1 Table: surveys

Column	Type	Description
id	BIGINT	Primary key

name	VARCHAR	Survey respondent name
age	INTEGER	Age
gender	VARCHAR	Gender
location	VARCHAR	Location
health_condition	VARCHAR	Reported health condition
survey_date	DATE	Date of survey
status	VARCHAR	Survey workflow status
created_by	BIGINT	User who created the survey

4.2 Table: users (planned)

Column	Type	Description
id	BIGINT	Primary key
username	VARCHAR	Login username
password	VARCHAR	Login password
role	VARCHAR	User role

4.3 Survey Status Workflow

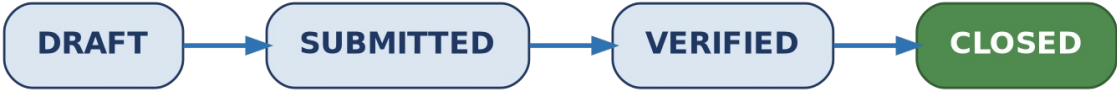


Figure 4: Survey status workflow

5. API List

The table below lists the full target API surface. Endpoints already implemented are marked **DONE**; the rest are **PLANNED** for upcoming weeks.

Method	Endpoint	Purpose	Status
GET	/surveys	View all surveys	DONE
GET	/surveys/{id}	View one survey	DONE
POST	/surveys	Create survey	DONE
PUT	/surveys/{id}	Update survey	PLANNED
GET	/surveys/search	Search surveys	PLANNED
PATCH	/surveys/{id}/status	Change survey status	PLANNED
GET	/dashboard	Dashboard summary	PLANNED

5.1 Currently Implemented — Details

GET /surveys

Returns the survey listing page.

GET /surveys/{id}

Returns details of a specific survey.

POST /surveys

Creates a new survey record. New records are assigned **DRAFT** status.

Requesting a survey by an ID that does not exist (e.g. `GET /surveys/9999`) currently returns a generic `Whitelabel` error page, since a `RuntimeException` is thrown. This is acceptable for the Week 3 foundation and will later be replaced with a proper `SurveyNotFoundException` and a friendly error page.

6. Working Local Setup

Steps to run the current Week 3 foundation locally.

6.1 Prerequisites

- Java JDK 17 or later
- Maven (or the bundled mvnw wrapper)
- MySQL server running locally

6.2 Database Setup

```
CREATE DATABASE community_health_survey;
```

6.3 Configure application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/community_health_survey
spring.datasource.username=root
spring.datasource.password=<your_mysql_password>
spring.jpa.hibernate.ddl-auto=update
server.port=8081
```

6.4 Build and Run

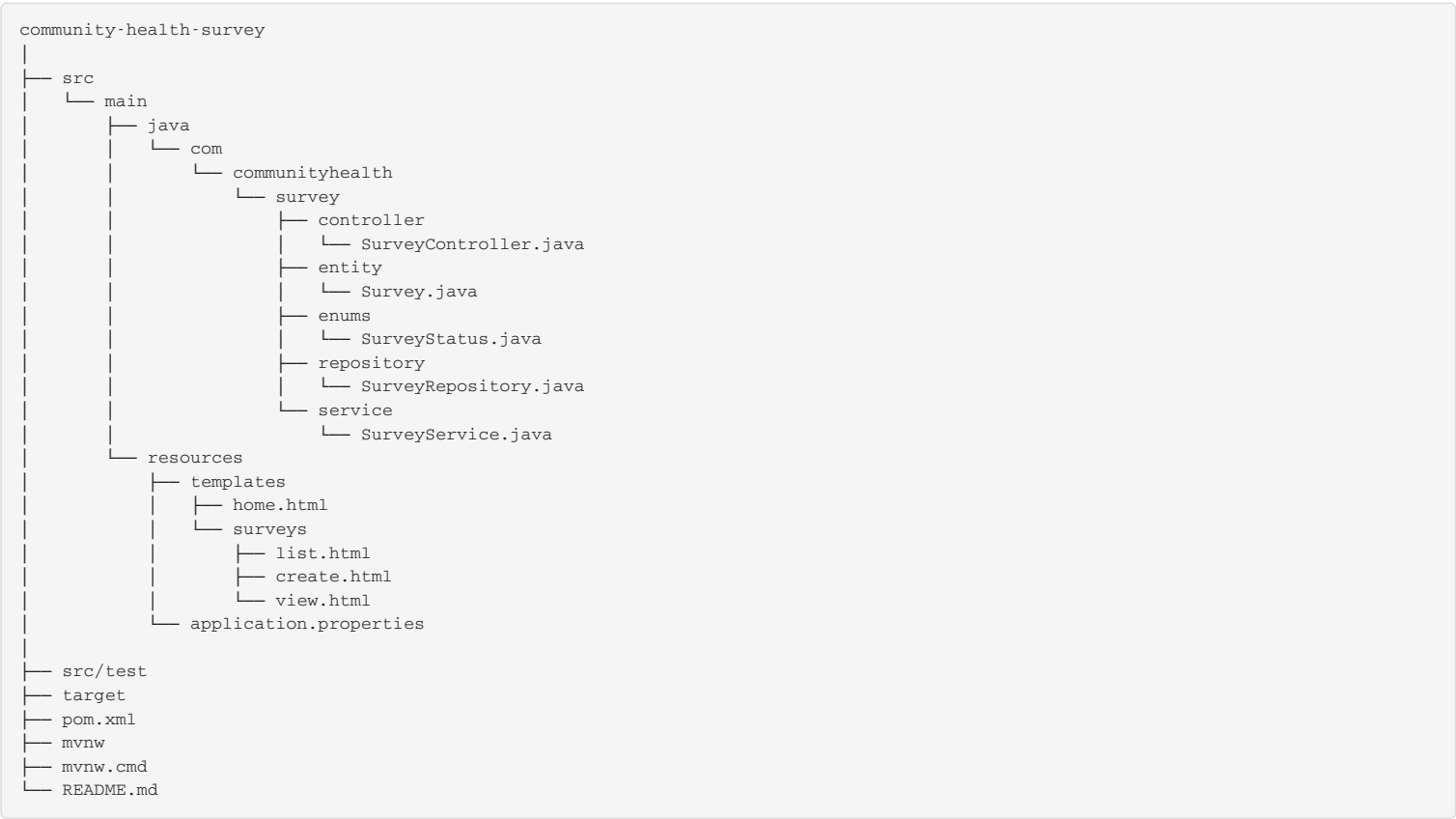
```
./mvnw clean package
./mvnw spring-boot:run
```

The application starts on http://localhost:8081 .

6.5 Verify

- Open http://localhost:8081/surveys to view the survey list.
- Use the create form to add a new survey record (saved with status DRAFT).
- Open http://localhost:8081/surveys/{id} to view a specific record.

6.6 Current Project Structure



7. Week 3 Deliverable Summary

The SRS summary, use-case diagram, architecture diagram, data model, API list and working local setup documented above complete the Week 3 deliverable. The local Spring Boot foundation — project skeleton, MySQL connection, and core Create/View survey functionality — is working and ready for Week 4, where Git/GitHub branching, commits, issues and pull requests will be introduced per the 15-week plan.